Assignment Tile: - No. 4_2

Create any interactive data visualization dashboard for any real-life application like weather dashboard and Student score Dashboard. Visualize data using charts and graphs (use libraries like Chart.js or D3.js)

Name: - Piyush Rajendra Waghmare

PRN: - 123B1B284

DIV: - D4

Objectives: -

- To design and develop a responsive, web-based data visualization dashboard.
- To learn and apply **D3.js** (Data-Driven Documents) for binding raw data to Scalable Vector Graphics (SVG).
- To implement modern UI/UX design principles, including CSS grid/flexbox layouts, responsive scaling, and interactive elements.
- To effectively visualize multiple data series (Temperature, Humidity, and Rainfall) on a single shared timeline.

Outcomes: -

- Successfully created a fully functional, production-ready weather dashboard.

- Implemented dynamic chart animations, including smooth line drawing and staggered element fading.
- Developed an interactive user experience where users can hover over data points to reveal specific values via custom tooltips.
- Integrated external assets (OpenWeatherMap icons) seamlessly into both the HTML DOM and the D3.js SVG canvas.

Theory Concept in brief: -

- **Data Binding (D3.js):** The core concept of mapping an array of data to visual DOM elements. Instead of manually drawing shapes, D3 calculates the coordinates based on the data provided.
- **Scales and Axes:** Using mathematical functions (like d3.scaleLinear and d3.scalePoint) to translate raw data values (e.g., 0-100%) into pixel coordinates on the screen.
- **SVG (Scalable Vector Graphics):** A markup language for describing two-dimensional graphics. Because it uses math rather than pixels, the dashboard remains crisp on any screen resolution.
- **Event Handling:** Utilizing JavaScript DOM events (like mouseover and mouseout) to trigger dynamic style changes and render tooltips in real-time.

Technologies Used: -

- Frontend: HTML5 (Structure), CSS3 (Styling/Layout).
- Logic: Vanilla JavaScript (ES6+).
- Library: Chart.js (CDN-linked) for rendering data.
- **API: OpenWeatherMap API (Current & 5-day Forecast** data).

Dataset Used: -

The project utilizes a mocked, static JavaScript array mimicking a typical JSON response from a weather API (such as OpenWeatherMap).

- **Size:** 7 records representing a weekly forecast (Monday to Sunday).
- **Attributes:**
    - day (String): Day of the week.
    - temp (Integer): Average temperature in Celsius.
    - humidity (Integer): Average humidity percentage.
    - rain (Integer): Average rainfall in millimeters.
    - icon (String): Weather condition code used to fetch the corresponding images.

Functional Description: -

- **KPI Summary Cards:** The top section features three quick-glance metric cards displaying the weekly averages alongside high-quality weather icons.
- **Multi-Line SVG Chart:** A central graph plotting three distinct lines for Temperature, Humidity, and Rain, utilizing d3.curveMonotoneX for smooth aesthetic curves.
- **Interactive Tooltips:** Hovering over any data point (the white dots) triggers a floating tooltip that displays the exact numerical value and unit for that specific day.
- **On-Load Animations:** The dashboard utilizes D3 transitions to draw the lines from left to right (stroke-dashoffset manipulation) and stagger the appearance of the X-axis weather icons.
- **Responsive Viewport:** The chart container uses a CSS padding-bottom trick combined with an SVG viewBox to ensure the graph scales perfectly across mobile, tablet, and desktop screens.

Program Code with output Screenshot: -

Weather2.html (using d3.js)

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>Pro Weather Dashboard with Images</title>
```

```
<script src="https://d3js.org/d3.v7.min.js"></script>
<link
href="https://fonts.googleapis.com/css2?family=Inter:wght@400;600;800&dis
play=swap" rel="stylesheet">


<style>
  * {
    box-sizing: border-box;
    margin: 0;
    padding: 0;
  }

  body {
    font-family: 'Inter', sans-serif;
    background: #f0f4f8;
    color: #2d3748;
    padding: 40px 20px;
    display: flex;
    flex-direction: column;
    align-items: center;
  }

  h1 {
    font-weight: 800;
    font-size: 2.2rem;
    margin-bottom: 30px;
    color: #1a202c;
```

```
    letter-spacing: -0.5px;
}
.kpi-container {
    display: flex;
    gap: 20px;
    width: 100%;
    max-width: 1000px;
    margin-bottom: 25px;
}

.kpi-card {
    background: #ffffff;
    flex: 1;
    padding: 20px 25px;
    border-radius: 20px;
    display: flex;
    align-items: center;
    gap: 15px;
    box-shadow: 0 10px 30px rgba(0, 0, 0, 0.04);
    transition: transform 0.2s ease;
}

.kpi-card:hover {
    transform: translateY(-5px);
}
```

```
.kpi-card img {
    width: 70px;
    height: 70px;
    background: #f7fafc;
    border-radius: 50%;
    object-fit: contain;
}

.kpi-data h3 {
    font-size: 1.8rem;
    font-weight: 800;
    color: #1a202c;
}

.kpi-data p {
    font-size: 0.9rem;
    font-weight: 600;
    color: #a0aec0;
    text-transform: uppercase;
    letter-spacing: 0.5px;
}

.card {
    background: #ffffff;
    width: 100%;
    max-width: 1000px;
```

```
    padding: 30px 40px 40px 40px;
    border-radius: 20px;
    box-shadow: 0 10px 40px rgba(0, 0, 0, 0.05);
}

.card h2 {
    font-weight: 600;
    font-size: 1.2rem;
    color: #718096;
    margin-bottom: 20px;
    text-transform: uppercase;
    letter-spacing: 1px;
}


.chart-container {
    position: relative;
    width: 100%;
    padding-bottom: 45%;
}

svg {
    position: absolute;
    top: 0;
    left: 0;
    width: 100%;
```

```
    height: 100%;
    overflow: visible;
}
.line {
    fill: none;
    stroke-width: 3.5;
    stroke-linecap: round;
    stroke-linejoin: round;
}

.temp-line { stroke: #ff6b6b; }
.humidity-line { stroke: #4ecdc4; }
.rain-line { stroke: #45b7d1; }

.dot {
    fill: white;
    stroke-width: 2.5;
    cursor: pointer;
}
.dot:hover { r: 7; }

.temp-dot { stroke: #ff6b6b; }
.humidity-dot { stroke: #4ecdc4; }
.rain-dot { stroke: #45b7d1; }

/* Axes & Gridlines */
```

```
.domain { display: none; }
.tick line { display: none; }
.tick text {
    font-size: 13px;
    fill: #a0aec0;
    font-family: 'Inter', sans-serif;
    font-weight: 600;
}
.grid-line {
    stroke: #e2e8f0;
    stroke-dasharray: 4 4;
}
.tooltip {
    position: absolute;
    background: rgba(26, 32, 44, 0.95);
    color: #fff;
    padding: 10px 15px;
    border-radius: 8px;
    font-size: 13px;
    font-weight: 600;
    pointer-events: none;
    opacity: 0;
    transform: translate(-50%, -100%);
    transition: opacity 0.2s ease;
    box-shadow: 0 4px 15px rgba(0,0,0,0.15);
    z-index: 10;
```

```
    }
    .tooltip span { font-weight: 400; color: #cbd5e0; }

    @media (max-width: 768px) {
        .kpi-container { flex-direction: column; }
    }
</style>
</head>

<body>

<h1>Weather Monitoring Dashboard</h1>

<div class="kpi-container">
    <div class="kpi-card">
        <img src="https://openweathermap.org/img/wn/01d@4x.png"
alt="Sunny">
        <div class="kpi-data">
            <h3>28°C</h3>
            <p>Avg Temp</p>
        </div>
    </div>
    <div class="kpi-card">
        <img src="https://openweathermap.org/img/wn/03d@4x.png"
alt="Humidity">
        <div class="kpi-data">
            <h3>60%</h3>
```

```
        <p>Avg Humidity</p>
      </div>
    </div>
    <div class="kpi-card">
      <img src="https://openweathermap.org/img/wn/10d@4x.png"
alt="Rain">
      <div class="kpi-data">
        <h3>13mm</h3>
        <p>Avg Rainfall</p>
      </div>
    </div>
</div>

<div class="card">
    <h2>Weekly Weather Report</h2>
    <div class="chart-container">
      <svg viewBox="0 0 900 400"></svg>
    </div>
</div>

<div class="tooltip" id="tooltip"></div>

<script>
const data = [
    {day: "Mon", temp: 28, humidity: 60, rain: 10, icon: "01d"}, // Sun
    {day: "Tue", temp: 32, humidity: 55, rain: 5, icon: "02d"},  // Few Clouds
    {day: "Wed", temp: 30, humidity: 65, rain: 20, icon: "10d"}, // Rain
```

```
    {day: "Thu", temp: 34, humidity: 70, rain: 35, icon: "11d"}, // Thunderstorm
    {day: "Fri", temp: 31, humidity: 62, rain: 15, icon: "09d"}, // Showers
    {day: "Sat", temp: 29, humidity: 58, rain: 8, icon: "04d"},  // Broken Clouds
    {day: "Sun", temp: 27, humidity: 50, rain: 2, icon: "01d"}   // Sun
];

const svg = d3.select("svg");
// Increased bottom margin to fit the images
const margin = {top: 20, right: 150, bottom: 80, left: 40};
const width = 900 - margin.left - margin.right;
const height = 400 - margin.top - margin.bottom;

const g = svg.append("g")
    .attr("transform", `translate(${margin.left},${margin.top})`);

const tooltip = d3.select("#tooltip");

const x = d3.scalePoint()
    .domain(data.map(d => d.day))
    .range([0, width])
    .padding(0.5);

const y = d3.scaleLinear()
    .domain([0, 100])
    .range([height, 0]);
```

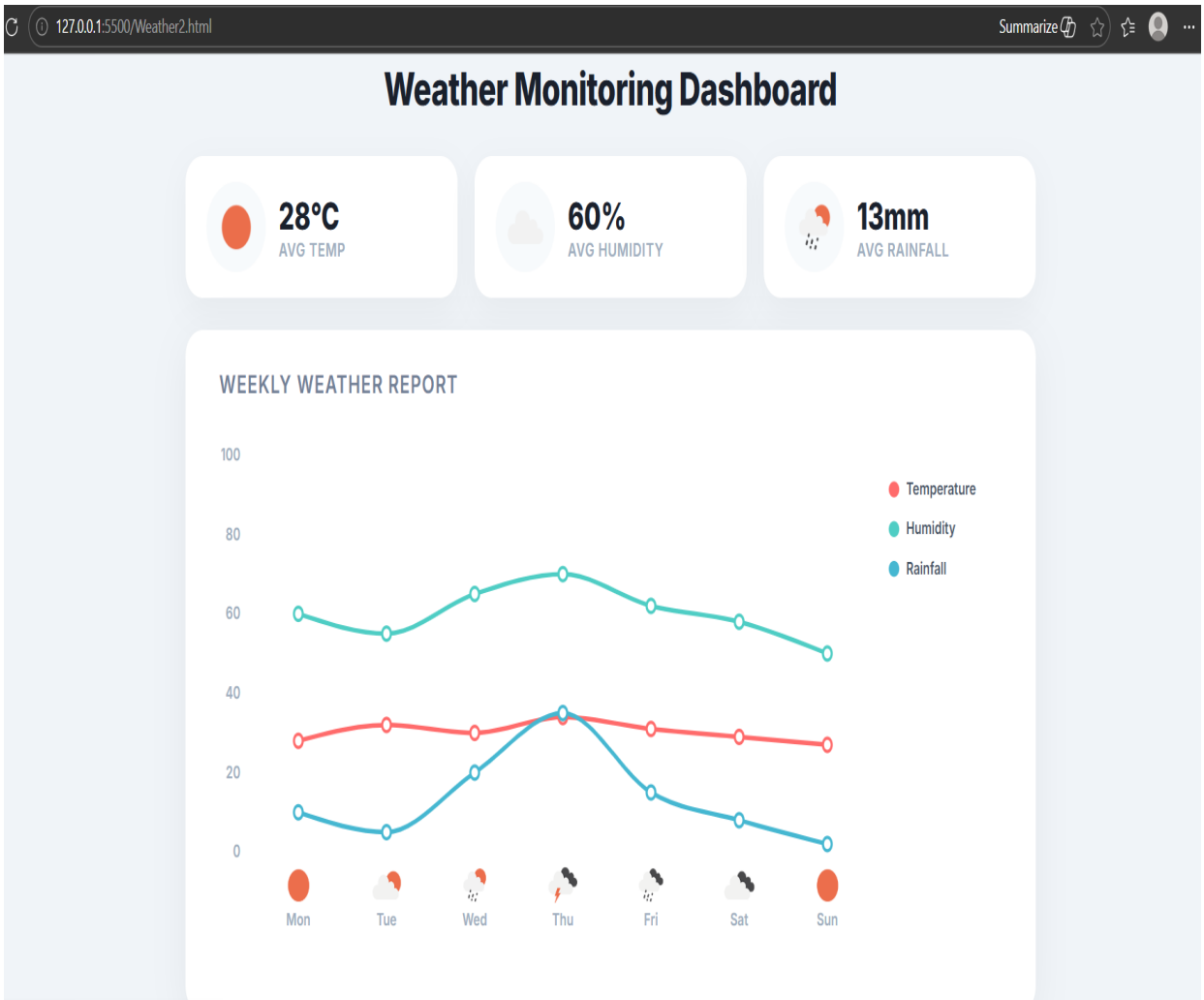
```
const yAxisGrid = d3.axisLeft(y).tickSize(-width).tickFormat('').ticks(5);
g.append('g').attr('class', 'grid-line').call(yAxisGrid);

g.append("g")
   .attr("transform", `translate(0,${height})`)
   .call(d3.axisBottom(x).tickPadding(40));

g.append("g").call(d3.axisLeft(y).ticks(5).tickPadding(10));

g.selectAll(".weather-icon")
   .data(data)
   .enter()
   .append("image")
   .attr("class", "weather-icon")
   .attr("xlink:href", d =>
`https://openweathermap.org/img/wn/${d.icon}@2x.png`)
   .attr("x", d => x(d.day) - 25) // Center the image over the tick
   .attr("y", height)            // Place it right below the graph line
   .attr("width", 50)
   .attr("height", 50)
   .style("opacity", 0)
   .transition()
   .delay((d, i) => i * 150 + 500) // Staggered fade-in animation
   .style("opacity", 1);

const createLine = (key) => d3.line().x(d => x(d.day)).y(d =>
y(d[key])).curve(d3.curveMonotoneX);
```

```
const lines = [
    { key: 'temp', class: 'temp-line', label: 'Temperature', unit: '°C' },
    { key: 'humidity', class: 'humidity-line', label: 'Humidity', unit: '%' },
    { key: 'rain', class: 'rain-line', label: 'Rainfall', unit: 'mm' }
];

lines.forEach(lineData => {
    const path = g.append("path")
        .datum(data)
        .attr("class", `line ${lineData.class}`)
        .attr("d", createLine(lineData.key));

    const totalLength = path.node().getTotalLength();
    path.attr("stroke-dasharray", totalLength + " " + totalLength)
        .attr("stroke-dashoffset", totalLength)
        .transition().duration(2000).ease(d3.easeCubicOut)
        .attr("stroke-dashoffset", 0);
});

lines.forEach(lineData => {
    g.selectAll(`.dot-${lineData.key}`)
        .data(data)
        .enter().append("circle")
        .attr("class", `dot ${lineData.key.split('-')[0]}-dot`)
        .attr("cx", d => x(d.day))
        .attr("cy", d => y(d[lineData.key]))
```

```
        .attr("r", 0)
        .on("mouseover", function(event, d) {
            d3.select(this).attr("r", 7);
            tooltip.transition().duration(200).style("opacity", 1);
            tooltip.html(`${lineData.label}:
<span>${d[lineData.key]}${lineData.unit}</span>`)
                .style("left", (event.pageX) + "px")
                .style("top", (event.pageY - 15) + "px");
        })
        .on("mouseout", function() {
            d3.select(this).attr("r", 5);
            tooltip.transition().duration(200).style("opacity", 0);
        })
        .transition().delay((d, i) => i * 200 + 500).duration(500).attr("r", 5);
});

const legend = svg.append("g")
    .attr("transform", `translate(${width + margin.left + 20}, ${margin.top + 20})`)
    .attr("font-size", "13px").attr("font-family", "Inter").attr("fill", "#4a5568");

const legendItems = [
    { label: "Temperature", color: "#ff6b6b" },
    { label: "Humidity", color: "#4ecdc4" },
    { label: "Rainfall", color: "#45b7d1" }
];

legendItems.forEach((item, i) => {
```

```
  const legendRow = legend.append("g").attr("transform", `translate(0, ${i * 30})`);

  legendRow.append("circle").attr("r", 6).attr("cx", 6).attr("cy", 6).attr("fill", item.color);

  legendRow.append("text").attr("x", 20).attr("y", 10).text(item.label).style("font-weight", "600");

});


</script>

</body>

</html>
```

Project Dashboard Output: -



Conclusion: -

This assignment successfully demonstrates the transition from raw, tabular data into an engaging and highly readable visual format. By combining the powerful data manipulation capabilities of D3.js with modern CSS styling and animations, we created a dashboard that is not only mathematically accurate but also visually appealing and user-friendly.